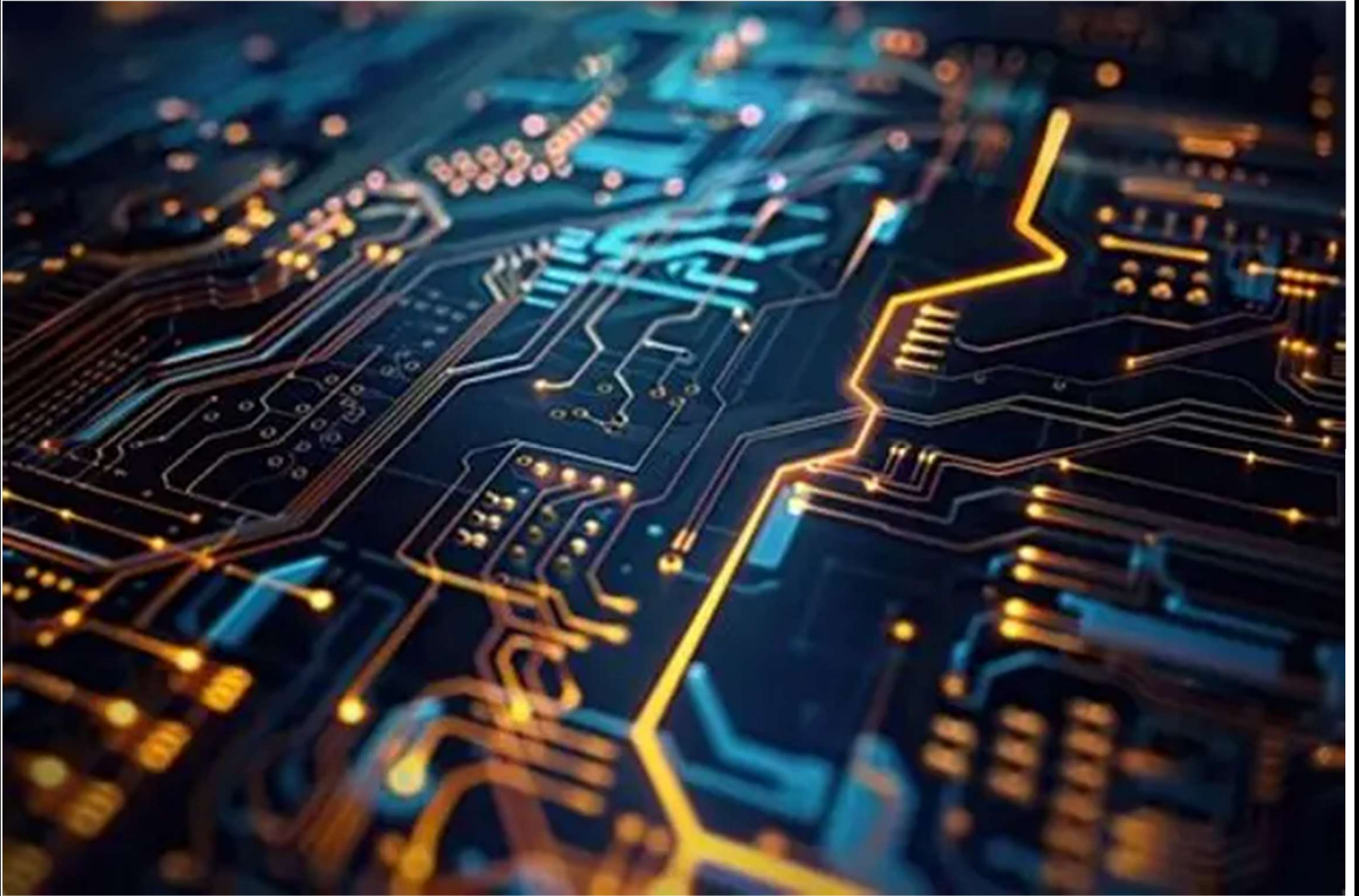
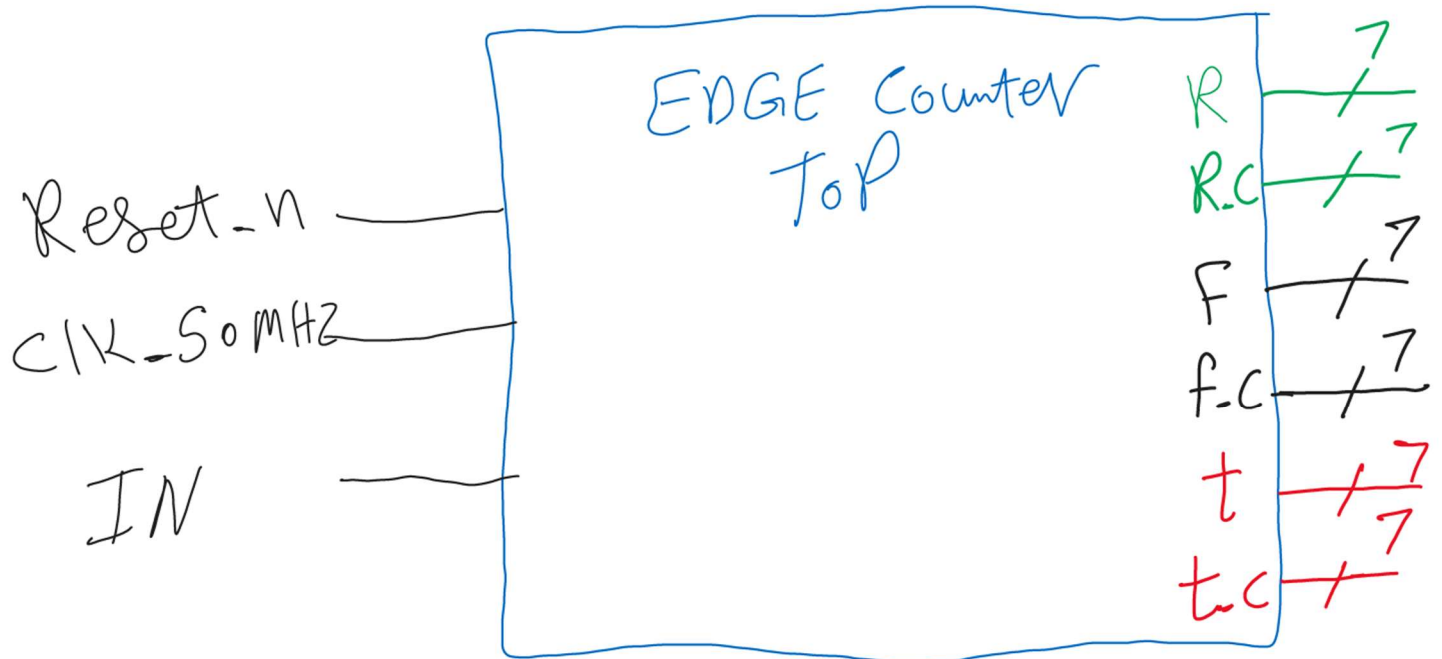
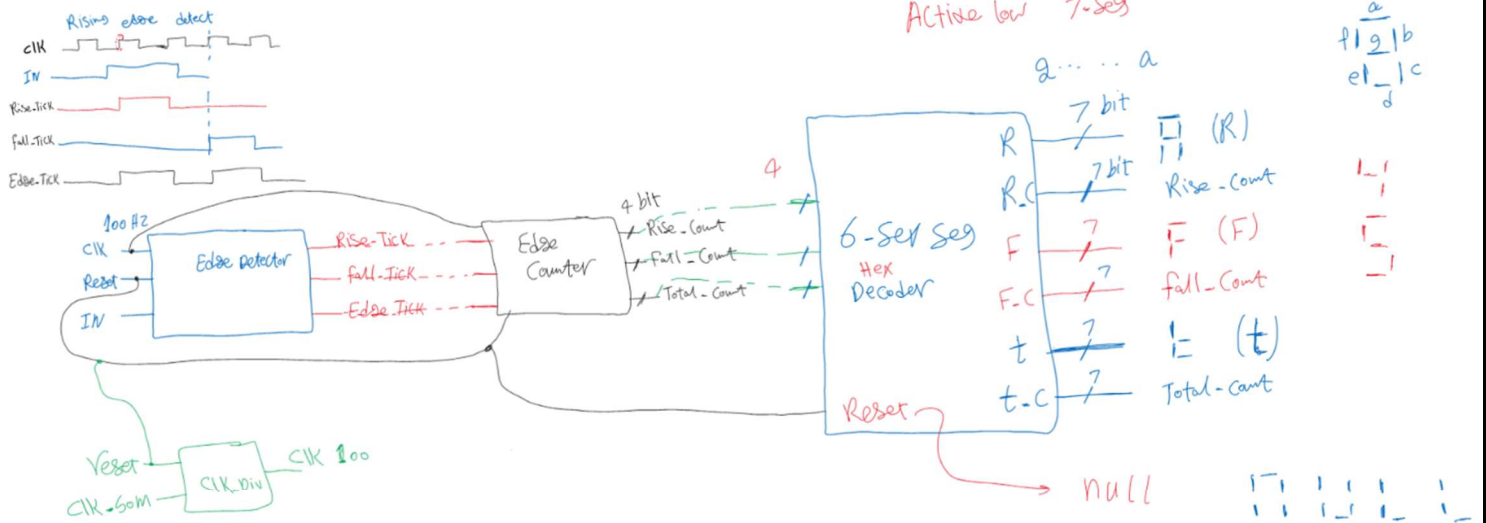


Assignment: Edge Counter



Name: Ahmed Wael Sadek El
Sawy

Assignment: Edge Counter



Assignment: Edge Counter

Clock Divider

clk_div_100hz.v

```
module clk_div_100hz #(
    parameter TOGGLE_COUNT = 250000
)(
    input  wire clk_50M,
    input  wire rst_n,
    output reg  clk_100Hz
);
    reg [$clog2(TOGGLE_COUNT)-1:0] counter;

    always @(posedge clk_50M or negedge rst_n) begin
        if (!rst_n) begin
            counter <= 0;
            clk_100Hz <= 1'b0;
        end else begin
            if (counter == TOGGLE_COUNT - 1) begin
                counter <= 0;
                clk_100Hz <= ~clk_100Hz;
            end else begin
                counter <= counter + 1;
            end
        end
    end
endmodule
```

tb_clk_div.v

```
module tb_clk_div;
    reg clk, rst_n;
    wire clk_out;
    integer errors = 0;

    // Use small count for fast simulation
    clk_div_100hz #(.TOGGLE_COUNT(4)) DUT (.clk_50M(clk), .rst_n(rst_n),
    .clk_100Hz(clk_out));

    initial begin clk = 0; forever #5 clk = ~clk; end

    initial begin
        rst_n = 0; @(negedge clk); rst_n = 1;

        @(negedge clk_out);
    end
endmodule
```

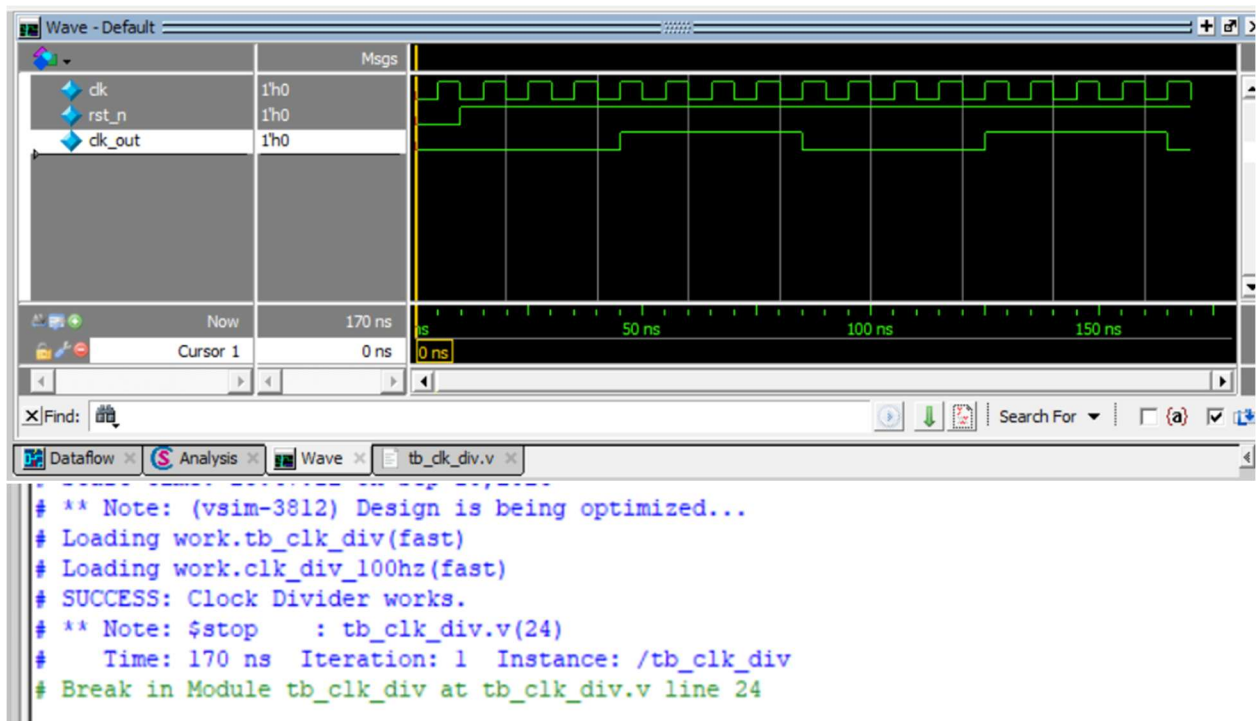
Assignment: Edge Counter

```
repeat(4) @(posedge clk);
@(negedge clk);
if(clk_out !== 1'b1) begin $display("ERROR: Did not toggle High.");
errors = errors + 1; end

repeat(4) @(posedge clk);
@(negedge clk);
if(clk_out !== 1'b0) begin $display("ERROR: Did not toggle Low."); errors
= errors + 1; end

if(errors == 0) $display("SUCCESS: Clock Divider works.");
$stop;
end
endmodule
```

Testing clk divider:



Assignment: Edge Counter

Edge Detector

edge_detector.v

```
module edge_detector (  
    input  wire clk_100Hz,  
    input  wire rst_n,  
    input  wire in_sig,  
    output reg  rise_tick,  
    output reg  fall_tick,  
    output reg  edge_tick  
);  
  
    parameter S_ZERO = 2'b00;  
    parameter S_RISE = 2'b01;  
    parameter S_ONE  = 2'b10;  
    parameter S_FALL = 2'b11;  
  
    reg [1:0] cs, ns;  
  
    // Next State Logic  
    always @(*) begin  
        case(cs)  
            S_ZERO: ns = in_sig ? S_RISE : S_ZERO;  
            S_RISE: ns = in_sig ? S_ONE  : S_FALL;  
            S_ONE:  ns = in_sig ? S_ONE  : S_FALL;  
            S_FALL: ns = in_sig ? S_RISE : S_ZERO;  
            default: ns = S_ZERO;  
        endcase  
    end  
  
    // State Memory (Async Active-Low Reset)  
    always @(posedge clk_100Hz or negedge rst_n) begin  
        if (!rst_n) cs <= S_ZERO;  
        else       cs <= ns;  
    end  
  
    // Moore Output Logic  
    always @(*) begin  
        rise_tick = (cs == S_RISE) ? 1'b1 : 1'b0;  
        fall_tick = (cs == S_FALL) ? 1'b1 : 1'b0;  
        edge_tick = (cs == S_RISE || cs == S_FALL) ? 1'b1 : 1'b0;  
    end  
endmodule
```

Assignment: Edge Counter

tb_edge_detector.v

```
module tb_edge_detector;
    reg clk, rst_n, in_sig;
    wire rise_tick, fall_tick, edge_tick;
    integer errors = 0;

    edge_detector DUT (.clk_100Hz(clk), .rst_n(rst_n), .in_sig(in_sig),
        .rise_tick(rise_tick), .fall_tick(fall_tick),
        .edge_tick(edge_tick));

    initial begin clk = 0; forever #5 clk = ~clk; end

    initial begin
        rst_n = 0; in_sig = 0;
        @(negedge clk); rst_n = 1; @(negedge clk);

        // Test Rising Edge
        in_sig = 1; @(negedge clk);
        if (!rise_tick || !edge_tick || fall_tick) begin $display("ERROR: Rise
Tick failed."); errors = errors+1; end

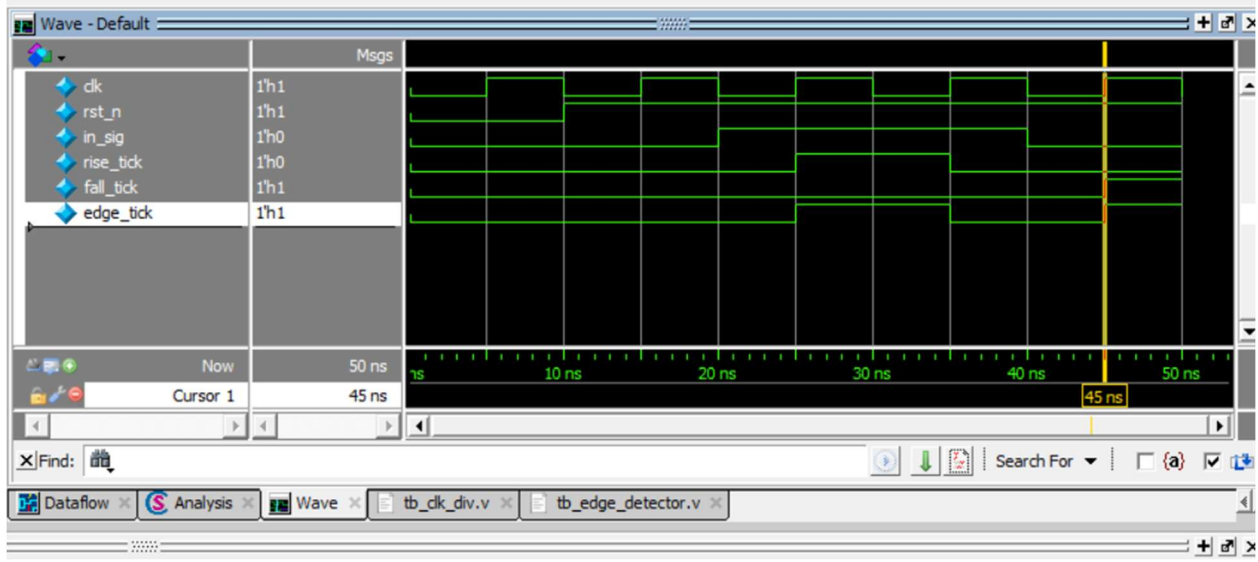
        @(negedge clk); // S_ONE state
        if (rise_tick || edge_tick) begin $display("ERROR: Tick lasted too
long."); errors = errors+1; end

        // Test Falling Edge
        in_sig = 0; @(negedge clk);
        if (!fall_tick || !edge_tick || rise_tick) begin $display("ERROR: Fall
Tick failed."); errors = errors+1; end

        if(errors == 0) $display("SUCCESS: Edge Detector works perfectly.");
        $stop;
    end
endmodule
```

Testing edge detector:

Assignment: Edge Counter



```
# ** Note: (vsim-3812) Design is being optimized...
# ** Note: (vopt-143) Recognized 1 FSM in module "edge_detector(fast)".
# Loading work.tb_edge_detector(fast)
# Loading work.edge_detector(fast)
# SUCCESS: Edge Detector works perfectly.
# ** Note: $stop      : tb_edge_detector.v(27)
#   Time: 50 ns   Iteration: 1   Instance: /tb_edge_detector
# Break in Module tb_edge_detector at tb_edge_detector.v line 27
```

Edge Counter

edge_counter.v

```
module edge_counter (
    input wire clk_100Hz,
    input wire rst_n,
    input wire rise_tick,
    input wire fall_tick,
    input wire edge_tick,
    output reg [3:0] rise_count,
    output reg [3:0] fall_count,
    output reg [3:0] total_count
);
    always @(posedge clk_100Hz or negedge rst_n) begin
        if (!rst_n) begin
            rise_count <= 0;
            fall_count <= 0;
            total_count <= 0;
        end
    end
```

Assignment: Edge Counter

```
        end else begin
            if (rise_tick) rise_count <= rise_count + 1;
            if (fall_tick) fall_count <= fall_count + 1;
            if (edge_tick) total_count <= total_count + 1;
        end
    end
endmodule
```

tb_edge_counter.v

```
module tb_edge_counter;
    reg clk, rst_n, r_tick, f_tick, e_tick;
    wire [3:0] rc, fc, tc;
    integer errors = 0;

    edge_counter DUT (.clk_100Hz(clk), .rst_n(rst_n), .rise_tick(r_tick),
        .fall_tick(f_tick),
        .edge_tick(e_tick), .rise_count(rc), .fall_count(fc),
        .total_count(tc));

    initial begin clk = 0; forever #5 clk = ~clk; end

    initial begin
        rst_n = 0; r_tick = 0; f_tick = 0; e_tick = 0;
        @(negedge clk); rst_n = 1; @(negedge clk);

        r_tick = 1; e_tick = 1; @(negedge clk); // Pulse
        r_tick = 0; e_tick = 0; @(negedge clk); // Clear

        if (rc !== 1 || tc !== 1 || fc !== 0) begin $display("ERROR on Rise
Count."); errors = errors+1; end

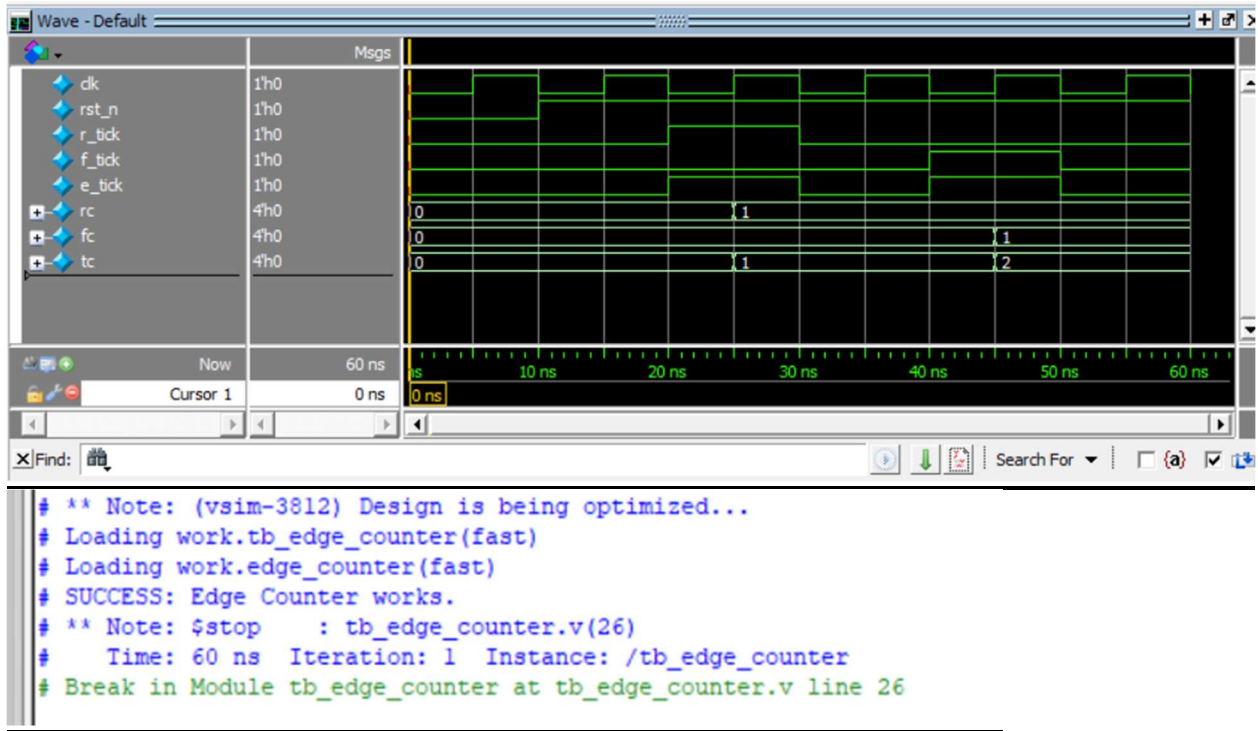
        f_tick = 1; e_tick = 1; @(negedge clk);
        f_tick = 0; e_tick = 0; @(negedge clk);

        if (fc !== 1 || tc !== 2 || rc !== 1) begin $display("ERROR on Fall
Count."); errors = errors+1; end

        if(errors == 0) $display("SUCCESS: Edge Counter works.");
        $stop;
    end
endmodule
```

Test edge counter:

Assignment: Edge Counter



6-Seven segment Display

hex2seg_single.v

```
module hex2seg_single (
    input  wire [3:0] hex,
    output reg  [6:0] seg
);
    // Active Low Encoding: 0 = ON, 1 = OFF
    always @(*) begin
        case (hex)
            4'h0: seg = 7'b1000000;
            4'h1: seg = 7'b1111001;
            4'h2: seg = 7'b0100100;
            4'h3: seg = 7'b0110000;
            4'h4: seg = 7'b0011001;
            4'h5: seg = 7'b0010010;
            4'h6: seg = 7'b0000010;
            4'h7: seg = 7'b1111000;
            4'h8: seg = 7'b0000000;
            4'h9: seg = 7'b0010000;
            4'hA: seg = 7'b0001000;
            4'hB: seg = 7'b0000011;
        endcase
    end
endmodule
```

Assignment: Edge Counter

```
        4'hC: seg = 7'b1000110;  
        4'hD: seg = 7'b0100001;  
        4'hE: seg = 7'b0000110;  
        4'hF: seg = 7'b0001110;  
        default: seg = 7'b1111111;  
    endcase  
end  
endmodule
```

hex_6seg_decoder.v

```
module hex_6seg_decoder (  
    input wire rst_n,  
    input wire [3:0] rise_count,  
    input wire [3:0] fall_count,  
    input wire [3:0] total_count,  
    output reg [6:0] seg_r, seg_rc,  
    output reg [6:0] seg_f, seg_fc,  
    output reg [6:0] seg_t, seg_tc  
);  
  
    // Internal wires to catch the outputs of the sub-modules  
    wire [6:0] out_rc, out_fc, out_tc;  
  
    // Instantiate for the Rise Count  
    hex2seg_single u_dec_rise (  
        .hex(rise_count),  
        .seg(out_rc)  
    );  
  
    // Instantiate for the Fall Count  
    hex2seg_single u_dec_fall (  
        .hex(fall_count),  
        .seg(out_fc)  
    );  
  
    // Instantiate for the Total Count  
    hex2seg_single u_dec_total (  
        .hex(total_count),  
        .seg(out_tc)  
    );  
  
    // Mux Logic: Choose between Reset state or Normal operation  
    always @(*) begin
```

Assignment: Edge Counter

```
    if (!rst_n) begin
        // "NULL" Reset Condition (Active Low)
        seg_r  = 7'b0101011;
        seg_rc = 7'b1000001;
        seg_f  = 7'b1000111;
        seg_fc = 7'b1000111;
        seg_t  = 7'b1111111;
        seg_tc = 7'b1111111;
    end else begin
        // Normal Operational Condition using the sub-module outputs
        seg_r  = 7'b0101111;
        seg_rc = out_rc;
        seg_f  = 7'b0001110;
        seg_fc = out_fc;
        seg_t  = 7'b0000111;
        seg_tc = out_tc;
    end
end
endmodule
```

Top module

top_edge_system.v

```
module top_edge_system #(
    parameter CLK_DIV_TOGGLE = 250000
)(
    input  wire clk_50M,
    input  wire rst_n,
    input  wire in_sig,
    output wire [6:0] seg_r, seg_rc,
    output wire [6:0] seg_f, seg_fc,
    output wire [6:0] seg_t, seg_tc
);
    wire clk_100Hz;
    wire r_tick, f_tick, e_tick;
    wire [3:0] r_cnt, f_cnt, t_cnt;

    // 1. Clock Divider
    clk_div_100hz #(.TOGGLE_COUNT(CLK_DIV_TOGGLE)) u_clk_div (
        .clk_50M(clk_50M), .rst_n(rst_n), .clk_100Hz(clk_100Hz)
    );
```

Assignment: Edge Counter

```
// 2. Edge Detector
edge_detector u_edge_det (
    .clk_100Hz(clk_100Hz), .rst_n(rst_n), .in_sig(in_sig),
    .rise_tick(r_tick), .fall_tick(f_tick), .edge_tick(e_tick)
);

// 3. Edge Counter
edge_counter u_edge_cnt (
    .clk_100Hz(clk_100Hz), .rst_n(rst_n),
    .rise_tick(r_tick), .fall_tick(f_tick), .edge_tick(e_tick),
    .rise_count(r_cnt), .fall_count(f_cnt), .total_count(t_cnt)
);

// 4. Hex Decoder
hex_6seg_decoder u_hex_dec (
    .rst_n(rst_n), .rise_count(r_cnt), .fall_count(f_cnt),
    .total_count(t_cnt),
    .seg_r(seg_r), .seg_rc(seg_rc), .seg_f(seg_f), .seg_fc(seg_fc),
    .seg_t(seg_t), .seg_tc(seg_tc)
);
endmodule
```

tb_edge_detector.v

```
module tb_top_system;
    reg clk_50M, rst_n, in_sig;
    wire [6:0] s_r, s_rc, s_f, s_fc, s_t, s_tc;
    integer errors = 0;

    // Instantiate with a toggle count of 2 for extremely fast simulation
    top_edge_system #(.CLK_DIV_TOGGLE(2)) DUT (
        .clk_50M(clk_50M), .rst_n(rst_n), .in_sig(in_sig),
        .seg_r(s_r), .seg_rc(s_rc), .seg_f(s_f), .seg_fc(s_fc), .seg_t(s_t),
        .seg_tc(s_tc)
    );

    // 50MHz Clock Generation
    initial begin clk_50M = 0; forever #5 clk_50M = ~clk_50M; end

    initial begin
        // Initialize Inputs
        rst_n = 0;
        in_sig = 0;

        // SCENARIO 1: Check initial Reset (nULL) State
    end
endmodule
```

Assignment: Edge Counter

```
@(negedge clk_50M);
if (s_r !== 7'b0101011 || s_rc !== 7'b1000001 || s_f !== 7'b1000111)
begin
    $display("ERROR [Scen 1]: Top module reset failed to display
'nULL'.");
    errors = errors + 1;
end else begin
    $display("PASS [Scen 1]: Reset state 'nULL' verified.");
end
rst_n = 1; // Release reset
repeat(10) @(posedge clk_50M); // Let the system stabilize

// SCENARIO 2: Single Pulse

in_sig = 1;
repeat(10) @(posedge clk_50M);
in_sig = 0;
repeat(10) @(posedge clk_50M);

// Rise=1 (7'b1111001), Fall=1 (7'b1111001), Total=2 (7'b0100100)
if (s_rc !== 7'b1111001 || s_fc !== 7'b1111001 || s_tc !== 7'b0100100)
begin
    $display("ERROR [Scen 2]: Single pulse counting failed.");
    errors = errors + 1;
end else begin
    $display("PASS [Scen 2]: Single pulse counted correctly.");
end

// SCENARIO 3: Multiple Rapid Pulses (3 pulses)

repeat(3) begin
    in_sig = 1; repeat(6) @(posedge clk_50M);
    in_sig = 0; repeat(6) @(posedge clk_50M);
end
repeat(5) @(posedge clk_50M); // Wait for processing

// Rise=4 (7'b0011001), Fall=4 (7'b0011001), Total=8 (7'b0000000)
if (s_rc !== 7'b0011001 || s_fc !== 7'b0011001 || s_tc !== 7'b0000000)
begin
    $display("ERROR [Scen 3]: Rapid multi-pulse counting failed.");
    errors = errors + 1;
end else begin
    $display("PASS [Scen 3]: Rapid multi-pulse counted correctly (R=4,
F=4, T=8).");
end
end
```

Assignment: Edge Counter

```
// SCENARIO 4: Long Pulse (Testing Level Rejection)

in_sig = 1;
repeat(50) @(posedge clk_50M); // Hold high for a very long time
in_sig = 0;
repeat(10) @(posedge clk_50M);

// Previous counts + 1 more pulse.
// Rise=5 (7'b0010010), Fall=5 (7'b0010010), Total=10 or Hex 'A'
(7'b0001000)
    if (s_rc !== 7'b0010010 || s_tc !== 7'b0001000) begin
        $display("ERROR [Scen 4]: Long pulse edge detection failed. (Possible
level-sensitivity issue).");
        errors = errors + 1;
    end else begin
        $display("PASS [Scen 4]: Long pulse correctly identified as single
edge (R=5, F=5, T=A).");
    end

// SCENARIO 5: Mid-Operation Reset

rst_n = 0;
repeat(2) @(posedge clk_50M);
if (s_r !== 7'b0101011 || s_rc !== 7'b1000001) begin
    $display("ERROR [Scen 5]: Mid-operation reset failed to clear
outputs.");
    errors = errors + 1;
end else begin
    $display("PASS [Scen 5]: Mid-operation reset cleared system
properly.");
end

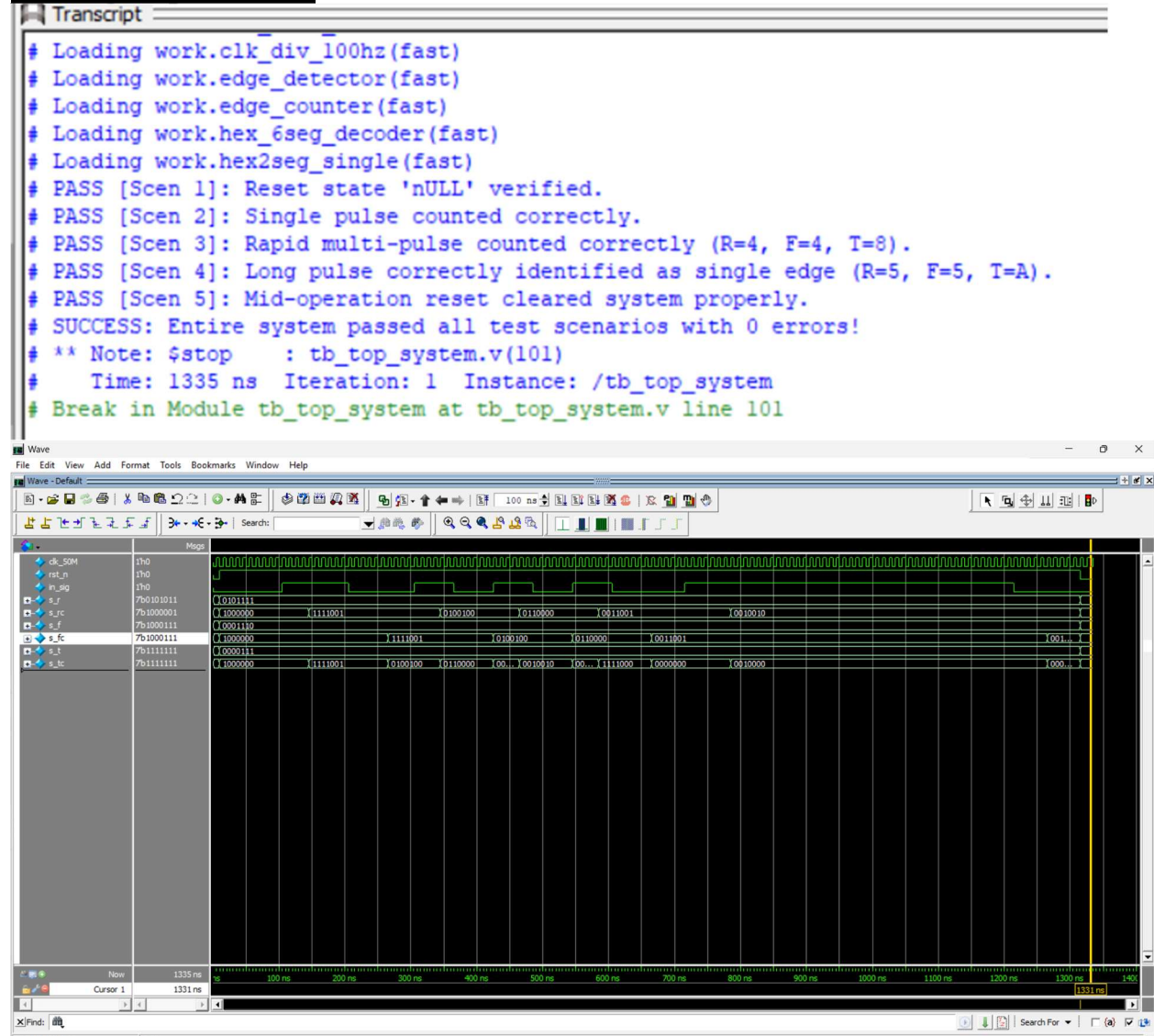
// Final Results

if(errors == 0)
    $display("SUCCESS: Entire system passed all test scenarios with 0
errors!");
else
    $display("FAILED: System finished with %0d errors.", errors);
```

Assignment: Edge Counter

```
$stop;  
end  
endmodule
```

Testing Top module:



Assignment: Edge Counter

